CENG454 HW3 - Core Breach: Pattern-Driven Systems Prototype

Full Name: Emir Evren
Student ID: 210444038
Course: CENG 454 - Game Programming
Homework Title: Project "Core Breach" - Pattern-Driven Systems Prototype
Project Name: ColorGuardian / Core Breach Prototype
GitHub Repository URL:
https://github.com/EmirEvren/CENG454-HW3-Emir-Evren-210444038
Submission Date: 3 May 2026

2. Gameplay Overview

For this homework I built a short single player defend the core prototype. The player controls a defender in a 3D dungeon arena and defends the chest from incoming enemies. I kept the visual scope reasonable, but I made sure the core loop is playble. The player can move, aim, shoot, toggle bullet color, grab ammo in the loot phase, and fight waves of enemies..

The loop is obviously two-phased. Each level starts with a loot phase where I fill the level with ammo pickups in varying colors. Once the timer runs out, enemies will spawn and combat will begin. In combat, the main rule is color matching. A bullet with a different color from an enemy cannot harm that enemy.So if u pick a different color enemys wont take damage. This means that ammo selection is part of the actual gameplay and not just a UI feature.

Win Condition: Clear all configured stages. The lose condition occurs if the player dies or the chest reaches zero health. I used the chest health bar, player health bar, stage text, phase text, timer, and color ammo UI so the player can understand what's happening without needing extra explanation.

I did not want to build a big game, but a small playable slice that already works as a real game system. As the project time was limited I first focused on making the main loop stable. The structure can still be expanded later with more stages, more enemy types and upgrades. Already in the current version we have repeated enemy pressure, repeated use of projectiles, damage to objectives, damage to the player, stage transitions and a final win or lose state.

3. Architecture Overview

I broke down the development process by separating the game into different sub-systems as opposed to grouping the rules into a larger manager. The main parts include controlling the player, shooting, ammunition management, enemy actions, level advancement, health management, user interface, object pooling, and system specific parts. In implementing the parts of the game, my goal was to facilitate interaction between the different sub-systems through events or interfaces where possible in order to minimize the dependency of one class on another.

3.1 Main Runtime Flow

1. The MainMenuUIManager launches the gameplay root when the user activates the Play button.

2. The StageManager controls the main stage loop, guiding the flow between loot phase, combat phase, stage clearance delay, and completion of the entire level.

3. The StageLootSpawner sets up the loot phase by spawning color-coded ammunition in the arena.

4. The EnemySpawner launches each wave of enemies, spawns enemies at spawn points, counts the number of active enemies, and signals when a wave is completed.

5. The EnemyController is responsible for enemy movement and attack timing. The target assignment is now delegated to the enemy target strategy instead of being hard-coded in the controller.

6. The PlayerShooter no longer creates a new bullet object every time the player fires. Instead, it asks for an appropriate bullet object from the BulletPool.

7. The BulletPool holds bullet objects of red, yellow, green, and blue colors. Once fired, it activates a bullet object that was inactive before.

8. The Bullet travels forward in space, stores its color and damage values, determines the color of the enemy upon collision, calculates its final damage based on the damage modifier chain, and returns to the pool once it has reached the end of its life cycle.

9. ChestHealth and PlayerHealth fire events when their health changes or they die. All UI systems and lose conditions subscribe to these events rather than accessing the classes directly.

10. The WinHandler listens for stage completion events and displays the winning screen after all set stages have been cleared.

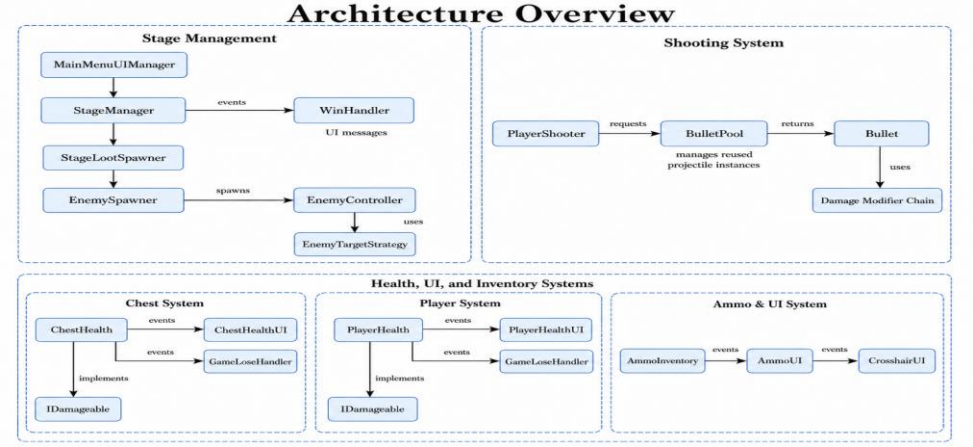
11. The GameLoseHandler listens for player death events and chest destruction events to handle the losing state.

3.2 Communication Structure

- Health classes publish events; they do not directly control their UI objects.
- AmmoInventory publishes ammo count and selected color changes, and both AmmoUI and CrosshairUI react to those events.
- StageManager publishes phase and stage-completion events.
- EnemyController uses a target strategy instead of hard-coded target selection branches.
- BulletPool owns the repeated projectile instances, while Bullet owns its own reset behavior through IPoolable.
- Bullet damage rules are built with a small Decorator chain, so new damage rules can be added without rewriting collision code.

3.3 Architecture Diagram Description

MainMenuUIManager -> StageManager -> StageLootSpawner / EnemySpawner. EnemySpawner creates enemies, and EnemyController uses an EnemyTargetStrategy to choose a target. PlayerShooter asks BulletPool for a projectile, and Bullet uses the damage modifier chain before applying damage through IDamageable. ChestHealth, PlayerHealth, AmmoInventory, and StageManager publish events. UI classes, WinHandler, and GameLoseHandler subscribe to those events. This keeps the systems connected, but not tightly locked to each other.



4. Pattern Justification Table

The table below maps the required patterns to the actual classes in the project. I wrote it to show where each pattern solves a gameplay or architecture problem, not only where a pattern name appears in code.

Pattern / Technique	Classes or Interfaces Involved	Problem Solved	Why I Used It Here
Interface	IDamageable; ChestHealth; PlayerHealth; EnemyHealth	Different objects can receive damage, but attackers should not depend on each concrete health script.	A bullet or enemy attack can call TakeDamage through one contract. The same call works for enemy, player, and chest/core health.
Interface	IPoolable; Bullet; BulletPool	Pooled bullets need a clear reset point before reuse.	The pool can tell each bullet when it is spawned and returned. This keeps timer, color, direction, and release state under control.
Interface	IEnemyTargetStrategy; EnemyTargetStrategy; PlayerPriorityTargetStrategy; CoreRushTargetStrategy	Enemy target choice should be replaceable per prefab.	EnemyController only asks a strategy for a target. I can change prefab behavior in the Inspector without rewriting the controller.
Interface	IWeaponDamageModifier; BaseWeaponDamageModifier; DamageModifierDecorator; ColorMatchDamageModifier; BonusDamageModifier	Damage rules should not all live inside Bullet.cs.	Each damage rule is a small modifier. This makes the weapon system easier to extend with future upgrades.
Observer	StageManager events; AmmoInventory events; ChestHealth events; PlayerHealth events; WinHandler; GameLoseHandler; UI scripts	Several systems need to react to the same gameplay change.	Events let UI, win logic, and lose logic react independently. The publisher does not directly control every reaction.
Strategy	EnemyController; PlayerPriorityTargetStrategy; CoreRushTargetStrategy	Fighter and archer enemies need different target behavior.	The fighter can prioritize the player, while the archer can rush the core. The owner class stays the same.
Object Pool	BulletPool; Bullet; IPoolable; PlayerShooter	Projectiles are created repeatedly during combat.	Bullets are preloaded, activated, reset, and reused. This avoids constant Instantiate/Destroy during shooting.
Decorator	IWeaponDamageModifier; DamageModifierDecorator; ColorMatchDamageModifier; BonusDamageModifier; Bullet	Weapon damage should support layered rules like color match and bonus damage.	The modifier chain lets me stack rules without changing Bullet collision code every time.

4.1 Pattern Design Notes

I tried to use the patterns where they actually fit the game, not just to put their names in the project. The Observer pattern made sense for health, ammo, and stage changes because more than one system needs to react to these events. For example, when health changes, the UI updates; when the player or chest dies, the lose screen appears; and when all stages are cleared, the win screen is shown.

For enemies, I used Strategy for target selection. I wanted different enemy prefabs to behave differently without writing a separate controller for each one. Fighter enemies can use a strategy that cares about the player, while archer enemies can use a strategy that focuses more on the chest. This way, EnemyController stays mostly responsible for movement and attacking, while the targeting rule can change.

For bullets, I used Object Pool because shooting happens many times during combat. Instead of creating and destroying a bullet every time, PlayerShooter gets a bullet from BulletPool. After the bullet hits something or its lifetime ends, it resets itself and goes back to the pool.

For the damage system, I used a small Decorator structure. The bullet damage starts from a base rule, then color matching and optional bonus damage can be added as separate layers. This keeps the bullet code cleaner and makes it easier to add more weapon effects later.

5. Screenshots and Evidence

The screenshots from the Unity project, code editor, and Github are included in this section. All screenshots will be kept in the document since they represent various aspects of the checklist.

5.1 Screenshot 1 - Playable Arena and Defendable Objective

Figure 1a. Combat phase view. The enemy is attacking, player health and chest health are visible, and the HUD shows the selected color ammunition.

Figure 1b. Loot phase view. The arena, chest/core, timer, stage text, and color ammunition UI are visible before combat starts.

These screenshots show that the core is not just decoration. The player has to protect it while also managing health, ammo color, and stage pressure. The combat screenshot is especially useful because both player health and chest health have changed, which proves that the gameplay loop is active.

5.2 Screenshot 2 - Custom Interface Evidence

IDamageable.cs

Assets > Scripts > Interfaces > IDamageable.cs

```
1   using UnityEngine;
2
3   public interface IDamageable
4   {
5       void TakeDamage(int amount);
6   }
```

Figure 2a. IDamageable defines the shared TakeDamage contract.

```
using UnityEngine;

public class EnemyHealth : MonoBehaviour, IDamageable
{
    private AmmoColor enemyColor = AmmoColor.Red;
```

Figure 2b. EnemyHealth implements IDamageable.

```
public class ChestHealth : MonoBehaviour, IDamageable
{
    [SerializeField] private int maxHealth = 500;
```

Figure 2c. ChestHealth implements IDamageable.

```
public class PlayerHealth : MonoBehaviour, IDamageable
{
    [SerializeField] private int maxHealth = 100;
```

Figure 2d. PlayerHealth implements IDamageable.

This evidence shows that damage is handled through a shared interface. Bullets and enemy attacks do not need to know whether the object is an enemy, the player, or the chest/core. They only need an IDamageable target. This reduced direct dependency between the attack systems and the concrete health classes.

5.3 Screenshot 3 - Observer-Style Event Flow

Assets > Scripts > Stage > StageManager.cs

```
9   {
29      [SerializeField] private float timerTextAppearDelay = 0.2f;
30
31      [Header("Stage Start Audio")]
32      [SerializeField] private AudioClip stage1StartSound;
33      [SerializeField] private AudioClip stage2StartSound;
34      [SerializeField] private AudioClip stage3StartSound;
35      [SerializeField] private AudioMixerGroup sfxMixerGroup;
36      [SerializeField, Range(0f, 1f)] private float stageStartSoundVolume = 1f;
37
38      [Header("Stage Settings")]
39      [SerializeField] private float lootDuration = 20f;
40      [SerializeField] private float stageClearDelay = 2f;
41
42      [SerializeField] private int[] enemiesPerStage = { 20, 30, 40 };
43      [SerializeField] private int[] lootPickupCountPerColor = { 1, 1, 2 };
44      [SerializeField] private int[] enemyDamagePerStage = { 10, 15, 15 };
45      [SerializeField] private int[] enemyHealthPerStage = { 1, 1, 3 };
46
47      public event Action<int> OnLootPhaseStarted;
48      public event Action<int> OnCombatPhaseStarted;
49      public event Action<int> OnStageCleared;
50      public event Action OnAllStagesCompleted;
51
```

Figure 3a. StageManager declares events for loot phase, combat phase, stage cleared, and all stages completed.

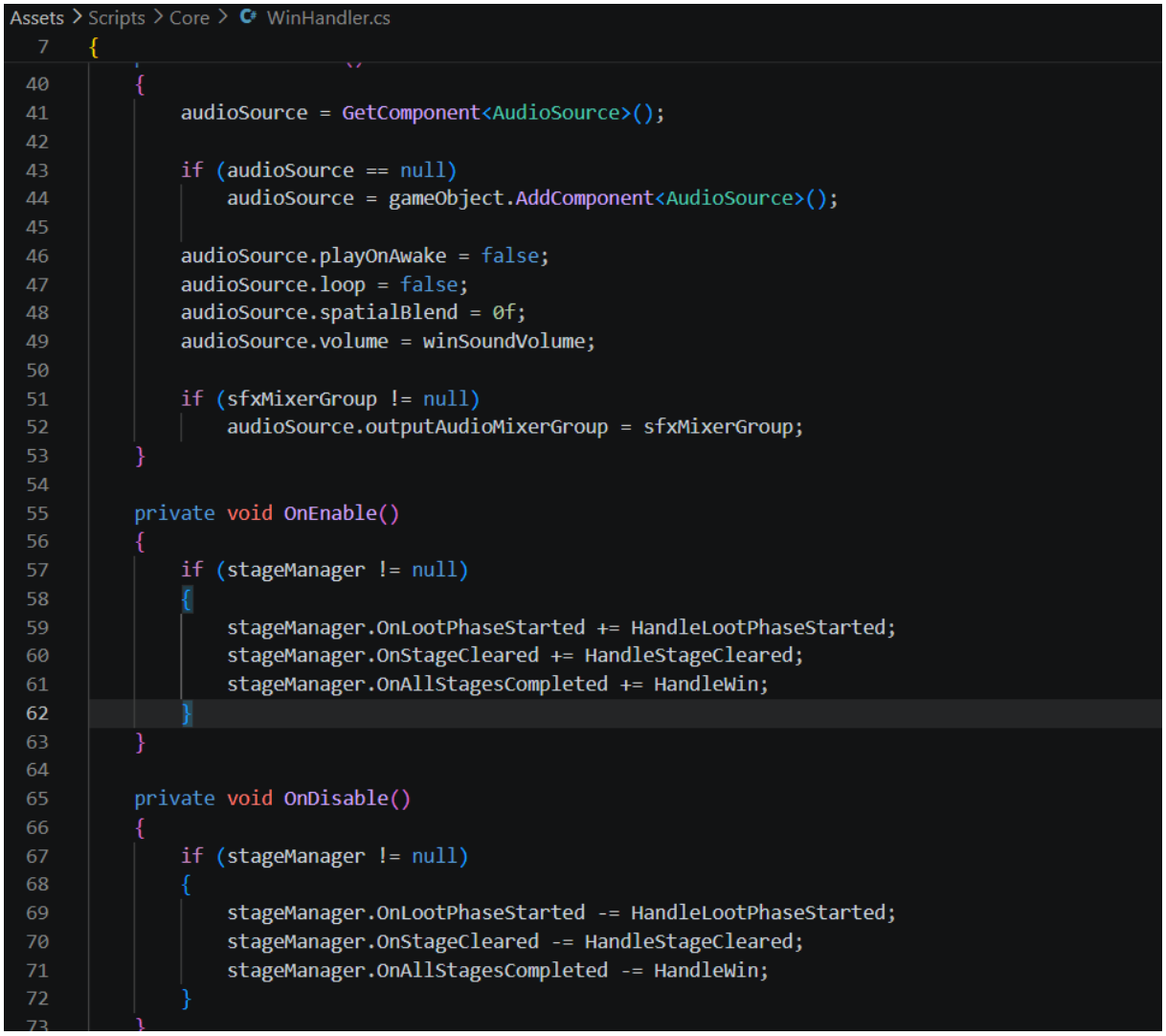


Figure 3b. WinHandler subscribes to StageManager events and also unsubscribes in OnDisable.

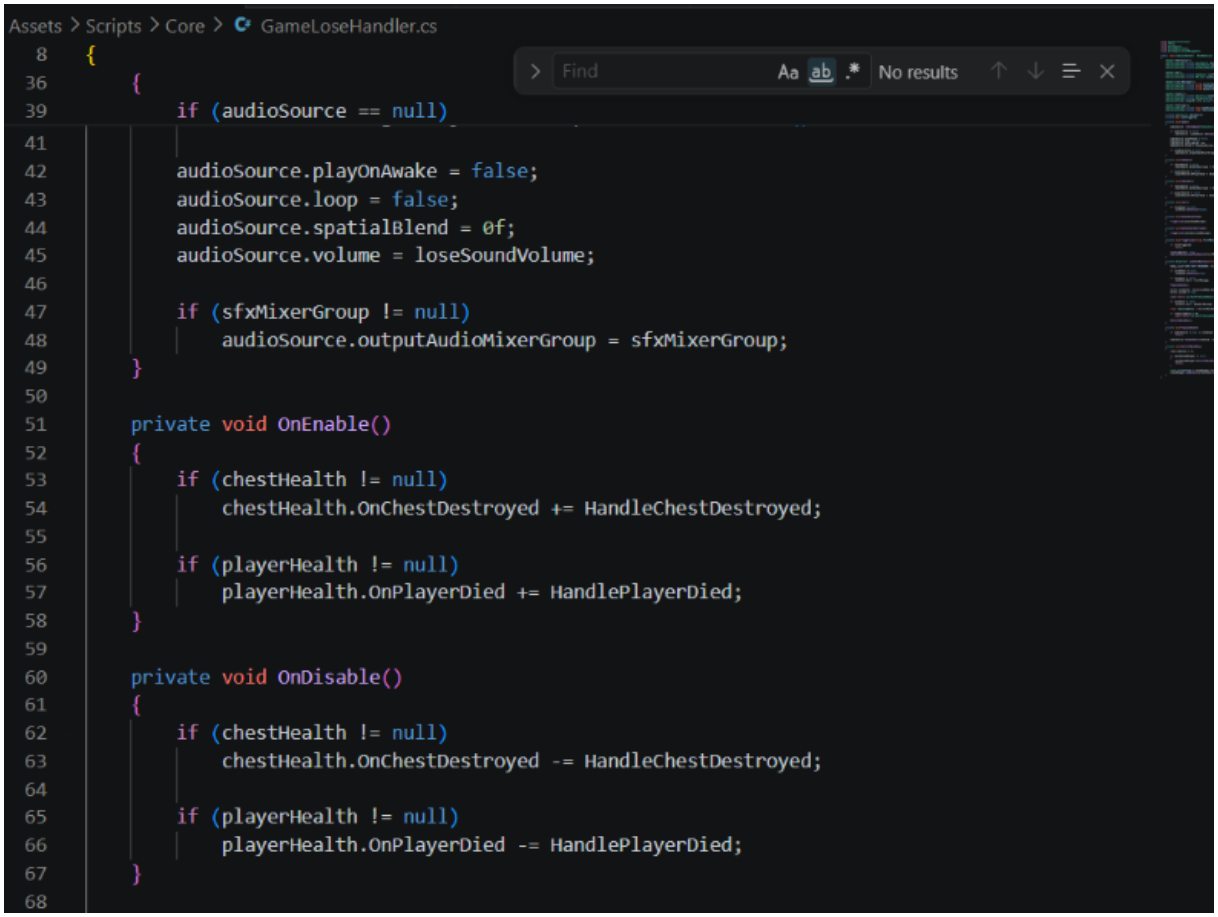


Figure 3c. GameLoseHandler subscribes to chest destroyed and player died events, then unsubscribes in OnDisable.

These screenshots show the Observer-style flow used in the project. StageManager publishes stage changes, but it does not directly control the win UI. WinHandler reacts to the events. In the same way, GameLoseHandler reacts to health/death events without PlayerHealth or ChestHealth directly controlling the lose panel. I also kept the unsubscribe code visible because lifecycle mistakes are a common source of event bugs.

5.4 Screenshot 4 - Strategy-Based Behavior Setup

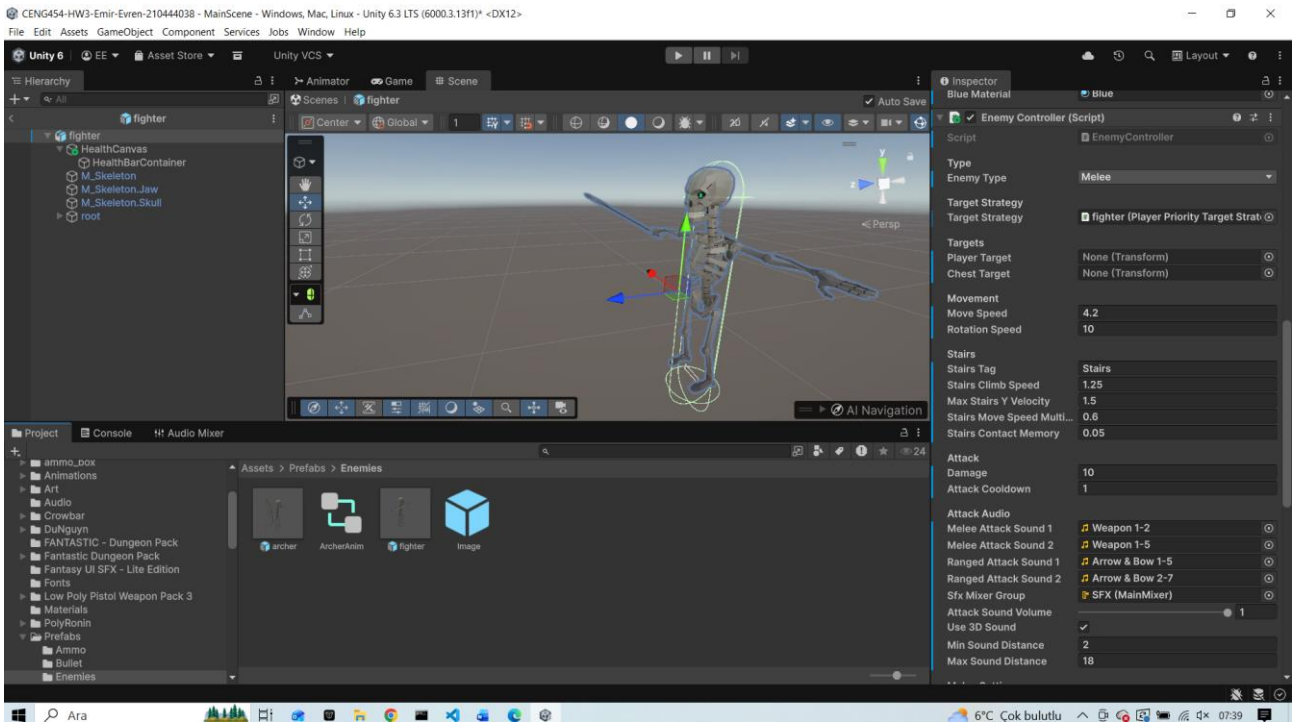


Figure 4a. Fighter prefab: EnemyController references PlayerPriorityTargetStrategy.

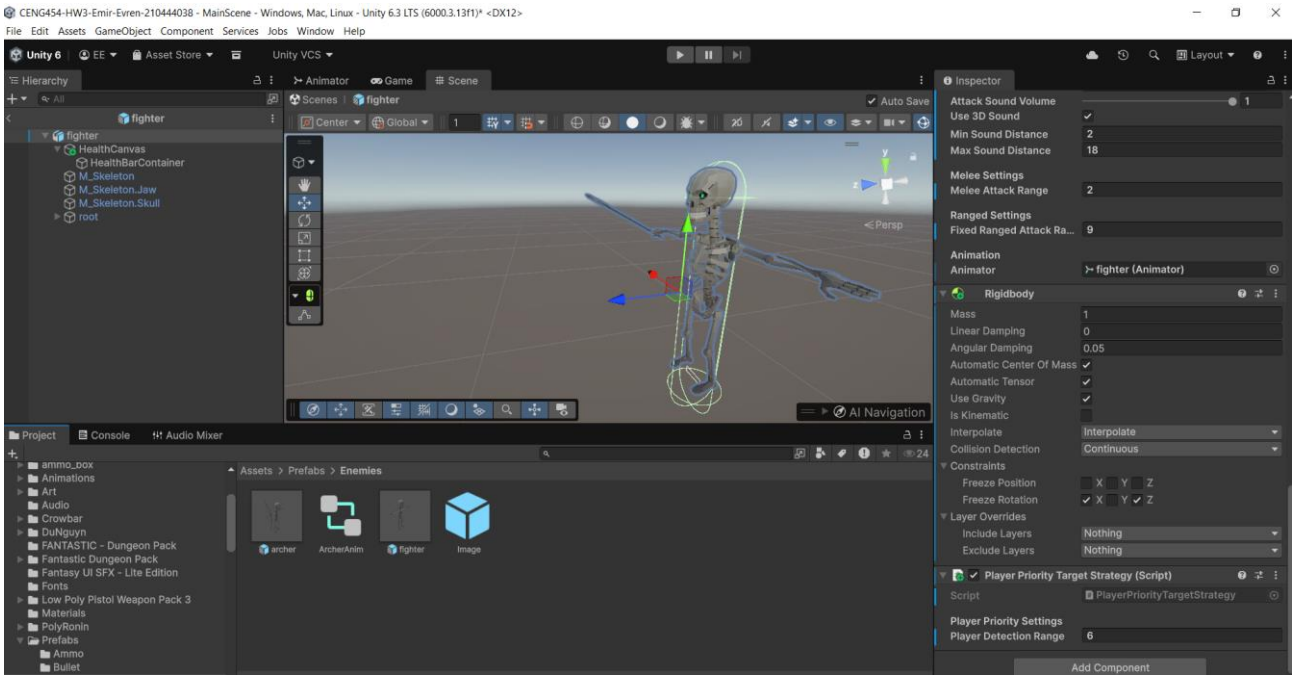


Figure 4b. Fighter prefab: PlayerPriorityTargetStrategy is attached and exposes its player detection range.

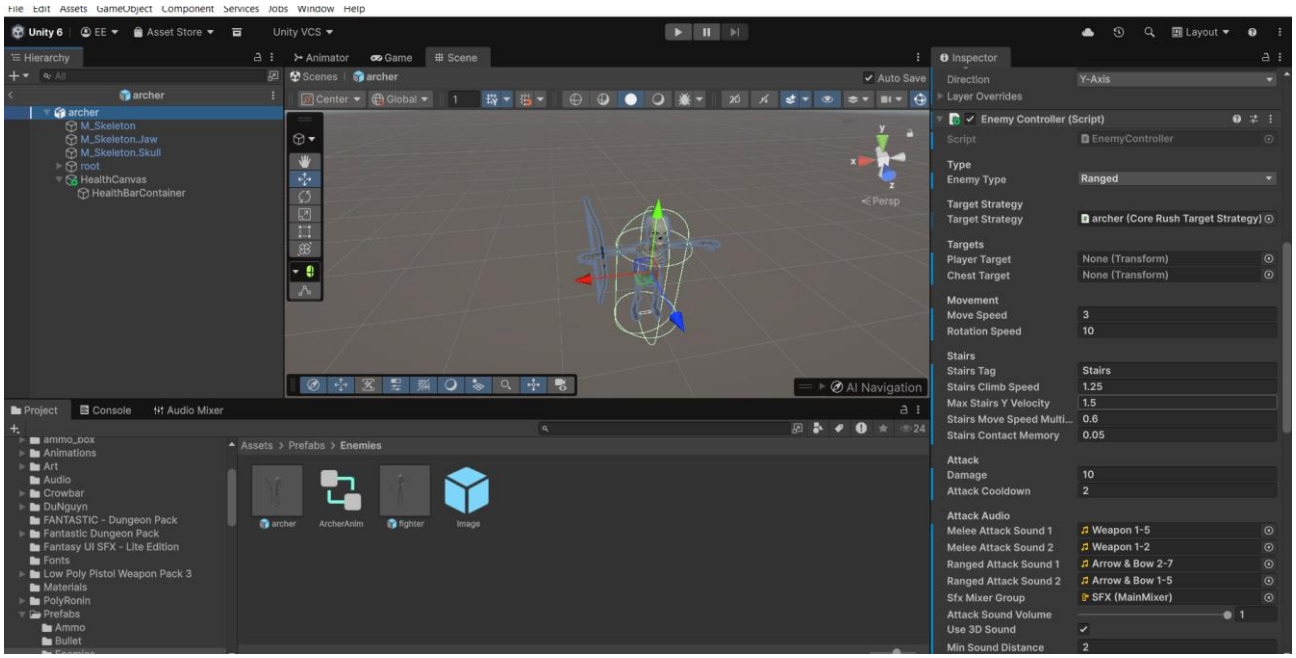


Figure 4c. Archer prefab: EnemyController references CoreRushTargetStrategy.

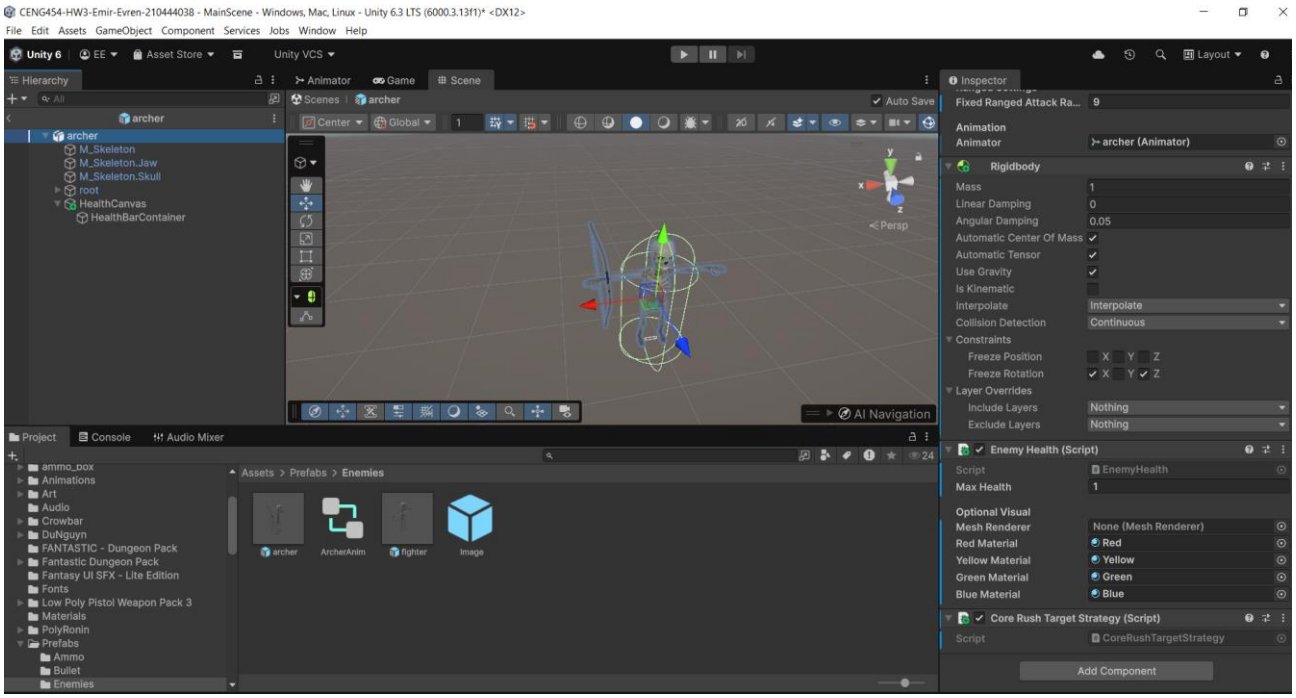


Figure 4d. Archer prefab: CoreRushTargetStrategy is attached.

This is the Strategy pattern setup. The fighter prefab can prioritize the player when the player is close, while the archer prefab is configured to rush the chest/core. EnemyController depends on the strategy interface, so I can change an enemy prefab behavior by assigning another strategy component instead of editing the controller script.

5.5 Screenshot 5 - Object Pool Runtime and Inspector Evidence

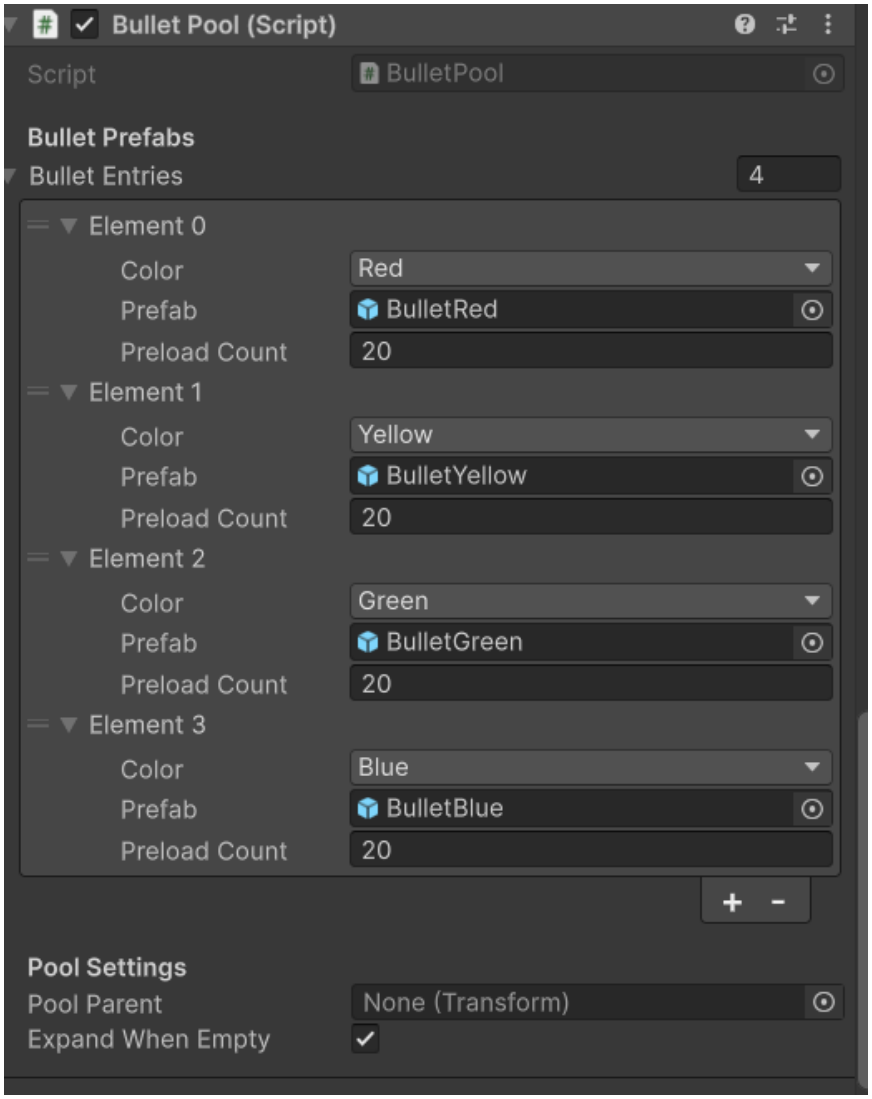


Figure 5a. BulletPool Inspector setup with four color entries and preload counts.

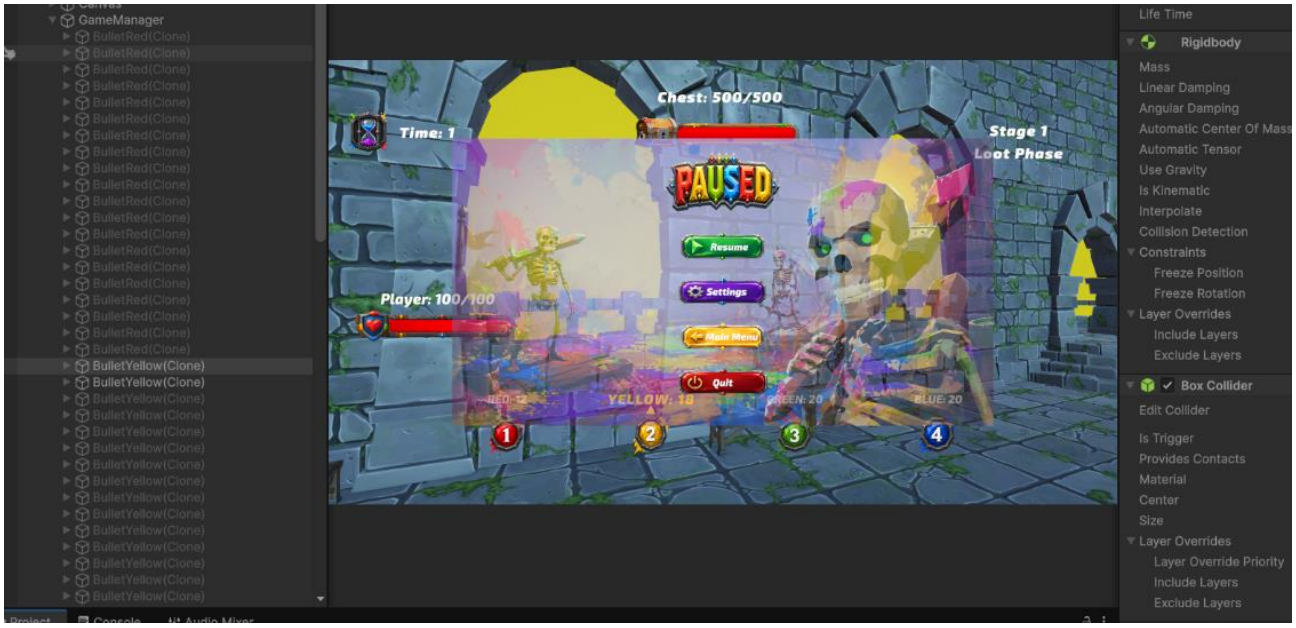


Figure 5b. Runtime hierarchy showing preloaded bullet instances under the pool.

The projectile system uses Object Pool because bullets are spawned repeatedly during combat. BulletPool preloads separate red, yellow, green, and blue projectile instances. When the player shoots, PlayerShooter asks the pool for a bullet. After impact or timeout, the bullet resets its state and returns to the pool instead of being destroyed.

5.6 Screenshot 6 - Decorator Usage Evidence

Assets > Scripts > Weapons > Bullet.cs

```
 4   {
 90      {
107      }
108
109      private void ApplyDamageToEnemy(EnemyHealth enemy)
110      {
111          if (enemy == null)
112              return;
113
114          if (damageModifier == null)
115              BuildDamageModifierChain();
116
117          int finalDamage = damageModifier.ModifyDamage(
118              damage,
119              ammoColor,
120              enemy.EnemyColor
121          );
122
123          if (finalDamage <= 0)
124              return;
125
126          enemy.TakeDamage(finalDamage);
127      }
128
129      private void BuildDamageModifierChain()
130      {
131          IWeaponDamageModifier modifier = new BaseWeaponDamageModifier();
132
133          if (requireColorMatch)
134              modifier = new ColorMatchDamageModifier(modifier);
135
136          if (useBonusDamageModifier && bonusDamage > 0)
137              modifier = new BonusDamageModifier(modifier, bonusDamage);
138
139          damageModifier = modifier;
140      }
```

Figure 6a. Bullet.cs builds and uses the weapon damage modifier chain before applying damage.

Assets > Scripts > Weapons > DamageModifiers > DamageModifierDecorator.cs

```
1   public abstract class DamageModifierDecorator : IWeaponDamageModifier
2   {
3       protected readonly IWeaponDamageModifier wrappedModifier;
4
5       protected DamageModifierDecorator(IWeaponDamageModifier wrappedModifier)
6       {
7           this.wrappedModifier = wrappedModifier;
8       }
9
10      public abstract int ModifyDamage(int baseDamage, AmmoColor bulletColor, AmmoColor targetColor);
11
12      protected int GetWrappedDamage(int baseDamage, AmmoColor bulletColor, AmmoColor targetColor)
13      {
14          if (wrappedModifier == null)
15              return baseDamage;
16
17          return wrappedModifier.ModifyDamage(baseDamage, bulletColor, targetColor);
18      }
19  }
```

Figure 6b. DamageModifierDecorator stores a wrapped IWeaponDamageModifier reference.

These screenshots show the Decorator-based weapon damage system. Bullet.cs starts with BaseWeaponDamageModifier and wraps it with ColorMatchDamageModifier and optionally BonusDamageModifier. This means the color-match rule and bonus-damage rule are separate layers. If I add another weapon upgrade later, I can add another modifier instead of rewriting the bullet collision logic.

5.7 Screenshot 7 - Repository and Workflow Evidence

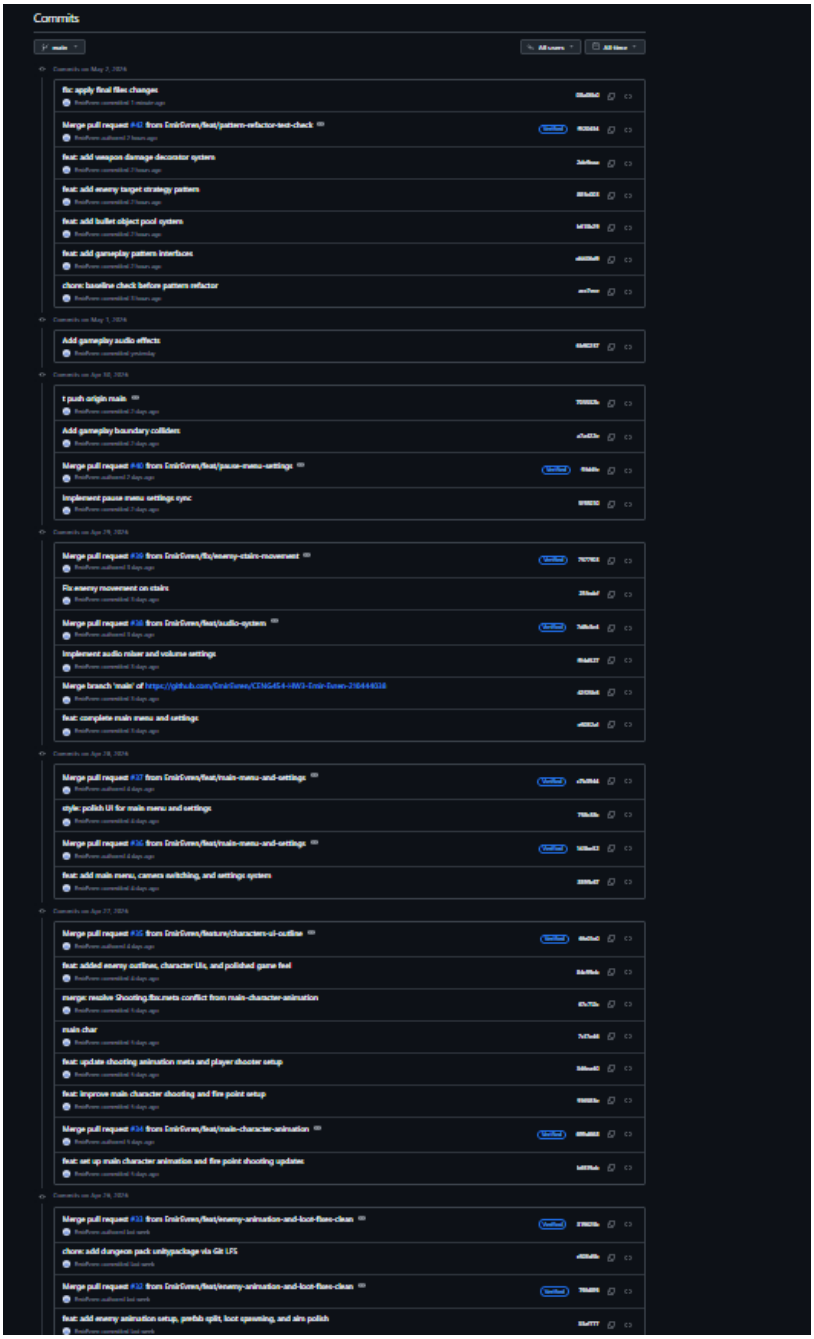


Figure 7a. GitHub commit history showing the pattern-refactor branch, pull request merge, and separate pattern commits.

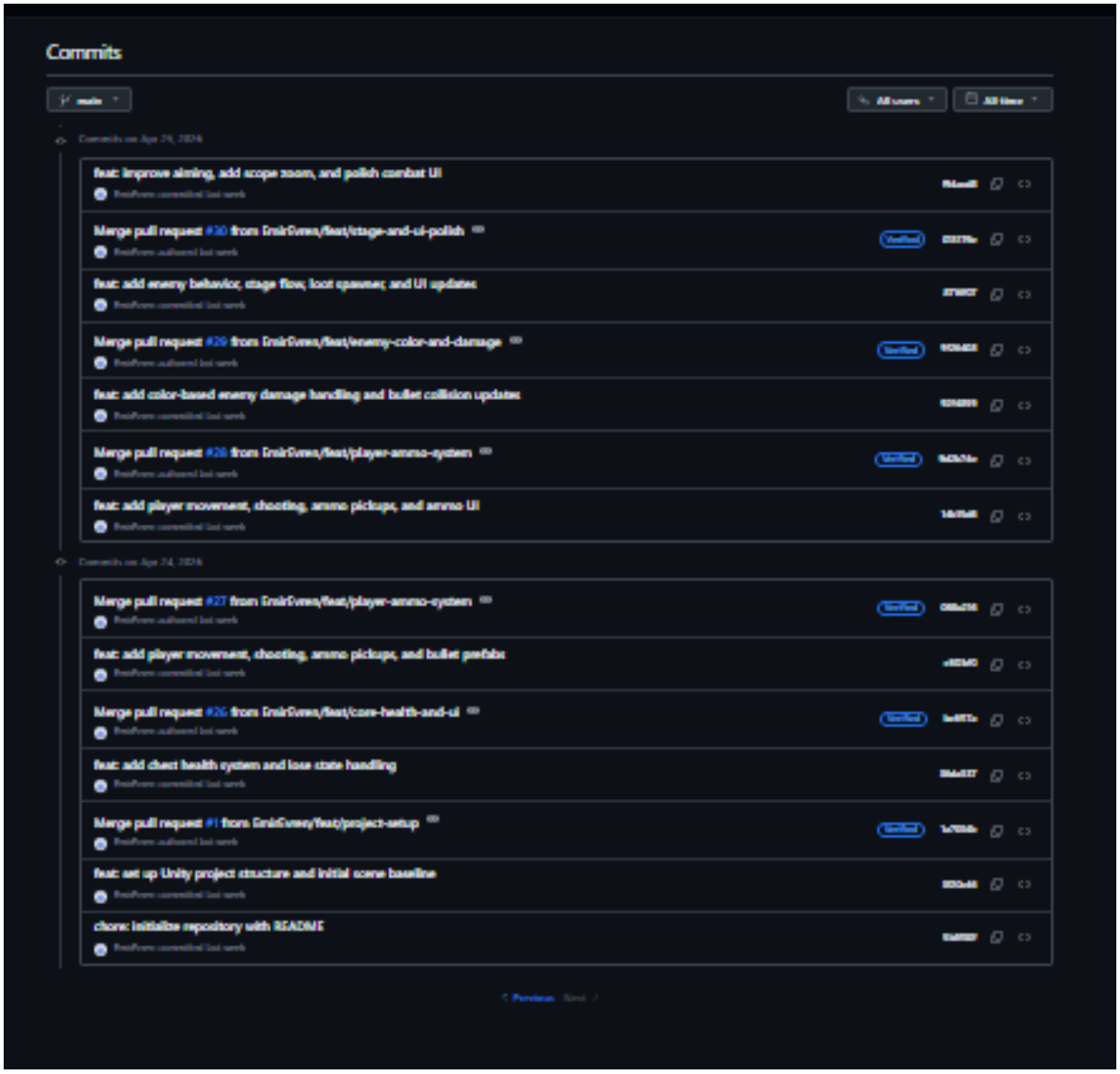


Figure 7b. Additional commit history showing earlier feature branches and merge evidence.

The repository evidence shows that the work was not submitted as one single final commit. The pattern work was split into commits for gameplay interfaces, bullet object pooling, enemy target strategy, and weapon damage decorator. The pattern-refactor branch was merged into main, which makes the process easier to review.

6. Debugging Story

One real bug happened while I was adding the Decorator-based damage system. I created the damage modifier classes first, but Unity started giving compile errors that said IWeaponDamageModifier could not be found. The error appeared in DamageModifierDecorator, BaseWeaponDamageModifier, ColorMatchDamageModifier, BonusDamageModifier, and Bullet. Since all of those files depended on the same interface, I knew the problem was probably not inside each modifier class separately.

I checked the first compiler error instead of changing random code. Then I checked the Interfaces folder and confirmed that Unity did not have a valid IWeaponDamageModifier.cs file with the exact interface declaration that the other classes expected. The fix was simple but important: I created the interface file with the correct name and the correct method signature, then waited for Unity to recompile.

After that, the compile errors disappeared and the bullet damage chain worked during testing. This bug was useful because it reminded me that interface-based design only works if the contract exists first and every dependent class uses the same method signature.

7. Troubleshooting Answer - Debug Report #003: Ghost Subscriber in a Pool

Symptoms include a single event causing multiple reactions in the same enemy. For instance, a single core-damaged event can result in two instances of identical sound playback, duplicate visual effects, or repeated logic in spite of only one damage event being fired. Another potential symptom includes an inactive pooled enemy reacting after being recycled back into the pool.

The root cause of this issue is an error in event subscription lifecycle management. This involves subscribing to core-damaged events when spawning or activating the enemy and failing to unsubscribe when dying or being recycled back into the pool. In cases where the same pooled object is used again, it gets subscribed again, allowing the object to receive the event multiple times.

This problem can be corrected by unsubscribing before recycling the object back into the pool. If the object subscribes through OnEnable or on Spawn from the pool, it should unsubscribe through OnDisable or on Return to the Pool. It is also recommended to ensure that the object is not double-subscribed without unsubscribing previous subscriptions.

Prevention strategy involves ensuring that all event subscriptions get unsubscribed in the same level of the lifecycle. Pooled objects should consider OnReturnedToPool as their cleanup stage where both gameplay and event subscription states get reset.

8. Retrospective

The best design decision that was made was using events and interfaces to separate the different components of the game. The use of events for health, ammunition, and stage transitions made the UI, win condition handler, and loss condition handler react without having the gameplay classes control them. The refactoring of the Bullet Pool made a lot of difference since there is a continuous fight in combat. Reusing the bullets is better than creating and destroying new ones each time a player shoots.

However, if the game were to be developed further, the focus would shift to improving the upgrade mechanism. Currently, the Decorator pattern allows the stacking of weapons with different capabilities. What can follow is making upgrades available through pickups that allow players to increase damage, penetration, slow targets, or make area attacks. It is also advisable to separate enemy attacks in the Strategy family to give players the chance to change how enemies attack.